

Rapport De Projet De Fin De Semestre

Spécialité : Ingénierie En Intelligence Artificielle

Du projet :

Analyse et Application du Package caret pour la Classification et la Régression

Étudiants :

Imane AMAAZ, Hiba SOFYAN, Imane BENCHRIF, Chantry OLANDA-EYIBA,
NTOTOMENGUEMA Neil-alain-lubin, Wiame ADNANE, Naima SAIDI, AIT TEMGHART Hind,
Zainab Jamil

Enseignant :

Pr. Abderazzak MOUIHA

Date : 20 décembre 2024

Année scolaire : 2024-2025

Table des matières

Résumé	2
0.1 Introduction	3
0.1.1 Contexte Général	3
0.1.2 Objectifs du Projet	3
0.2 Description du Package Caret	3
0.2.1 Installation du package caret	4
0.3 Apprentissage Classification et Régression	5
0.3.1 Processus de Régression	5
0.3.2 Processus de classification	7
0.4 Classification	10
0.4.1 Description de 5 modèles de classification	10
0.4.2 Le jeu de données de la maladie d'Alzheimer	15
0.4.3 Prétraitement des données	18
0.4.4 Entraînement et Comparaison entre les modèles	23
0.4.5 Entraînement du modèle SVM	24
0.4.6 Entraînement du modèle XGBoost	25
0.4.7 Évaluation des Modèles	28
0.4.8 Conclusion	33
0.5 Régression	35
0.5.1 Description de 5 modèles de régression	35
0.5.2 Le jeu de données de prédiction des prix des maisons	40
0.5.3 Pré-traitement du jeu de données	41
0.5.4 Comparaison entre les modèles	43
0.5.5 Modèle Choisi : Régression Linéaire Robuste (RLM)	43
0.5.6 Conclusion	47
Conclusion Générale	49

Résumé

R est un langage couramment utilisé en statistiques et en science des données, offrant une vaste gamme de packages pour l'analyse et la modélisation. Parmi ces outils, le package **caret** (Classification and Regression Training) simplifie l'entraînement et l'évaluation des modèles de machine learning en centralisant l'accès à différents algorithmes de classification et de régression.

Dans le cadre du module R, ce rapport propose une étude de cas sur le package **caret**. L'accent est mis sur les méthodes de classification et de régression à travers l'application pratique de plusieurs modèles sur des jeux de données appropriés. Après une exploration des différentes techniques disponibles dans le package, une comparaison des performances est effectuée en utilisant plusieurs métriques.

Cette étude permet de mettre en exergue les algorithmes construits autour du package **caret**, en illustrant leur efficacité dans des tâches de classification et de régression.

Mots-clés : Langage R, Statistiques, Package caret, Machine Learning, Classification, Régression, Jeux de données, Métriques.

0.1 Introduction

0.1.1 Contexte Général

L'analyse de données et la modélisation prédictive occupent une place importante dans de nombreux domaines d'application. Le développement de modèles de **machine learning** (apprentissage automatique) permet de traiter des volumes massifs de données et d'extraire des informations pertinentes pour prédire des résultats futurs ou bien classer des observations en différentes catégories. Cependant, la création de modèles efficaces nécessite l'utilisation d'outils performants pour la gestion des données, la sélection des algorithmes et l'évaluation des résultats.

Dans le cadre de ce projet, nous nous intéressons au package **caret** de **R**, un outil puissant pour construire des modèles de régression et de classification.

0.1.2 Objectifs du Projet

Le principal objectif de ce projet est de se familiariser avec les fonctionnalités du package **caret** et de démontrer son utilisation à travers deux tâches majeures, la **régression** et la **classification**.

Pour ce faire, nous allons accomplir les étapes suivantes :

1. Appliquer des techniques de prétraitement à un dataset pour la régression et un autre pour la classification.
2. Comparer cinq modèles pour chaque type de tâche (régression et classification).
3. Mettre en œuvre des modèles prédictifs en utilisant les jeux de données prétraités.
4. Sélectionner les modèles avec les meilleures performances pour chaque tâche et les étudier en profondeur afin de mieux comprendre leur fonctionnement et leurs spécificités.

À travers ces différentes étapes, ce projet permettra d'acquérir une solide expérience dans l'analyse de données et l'utilisation de modèles de machine learning pour résoudre des problèmes de classification et de régression, tout en se basant sur le package **caret** comme outil central.

0.2 Description du Package Caret

Le package **caret** (Classification And Regression Training) est une bibliothèque **R** complète conçue pour rationaliser et standardiser le processus de

création de modèles prédictifs. Développé par **Max Kuhn** et lancé en **2007**, caret regroupe un ensemble de fonctions permettant d'automatiser et de simplifier diverses étapes du flux de travail en modélisation, telles que :

- **Division des données** : Facilite le partitionnement des ensembles de données en sous-ensembles d'entraînement et de test.
- **Prétraitement des données** : Inclut des techniques de transformation telles que la normalisation, l'imputation des valeurs manquantes et la réduction de dimensions.

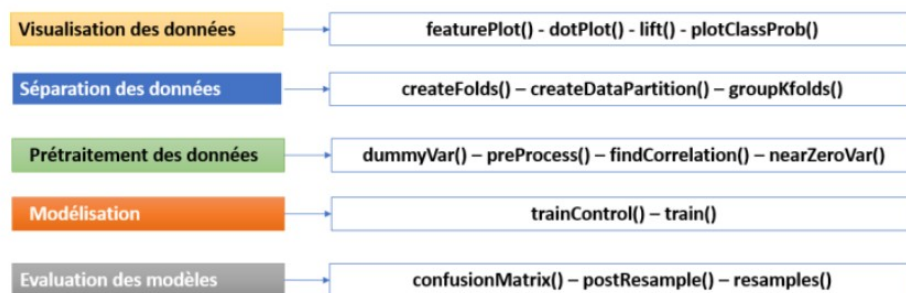


FIGURE 1 – Fonctions principales du package caret

- **Réglage des modèles** : Automatise l'optimisation des hyperparamètres à l'aide de techniques de rééchantillonnage, telles que la validation croisée, afin d'améliorer la robustesse et la précision des modèles.
- **Évaluation des performances** : Calcul de métriques (matrice de confusion, précision, etc...).

0.2.1 Installation du package caret

Le processus de l'installation du package caret, inclut les étapes suivantes :

1. Exécution de la commande dans R ou RStudio :

```
> install.packages("caret")
```

2. Chargement du package :

```
> library(caret)
```

3. Vérification de l'installation :

```
> packageVersion("caret")
```

0.3 Apprentissage Classification et Régression

Dans cette section, nous explorons les deux principales approches utilisées en apprentissage supervisé : **la régression** et **la classification**. Ces techniques permettent de résoudre différents types de problèmes prédictifs. Bien que les deux impliquent de faire des prévisions, elles diffèrent par leurs objectifs et la nature des résultats qu'elles produisent.

0.3.1 Processus de Régression

La régression est utilisée pour prédire une variable numérique continue en fonction d'une ou plusieurs variables explicatives.

Étapes principales :

1. **Collecte des données** : Pour effectuer une régression, des données sont collectées où la variable cible est continue, et les variables explicatives sont utilisées pour prédire cette variable cible.
2. **Modélisation** : L'algorithme de régression apprend à partir des données d'entraînement pour déterminer une fonction mathématique (souvent une droite ou une courbe) qui correspond aux données. Le modèle peut être :
 - **Linéaire** : lorsque la relation entre les variables est représentée par une droite. Un graphique typique pour une régression linéaire montre les données sous forme de points et une ligne qui représente la fonction prédictive du modèle. La ligne est ajustée de manière à minimiser la

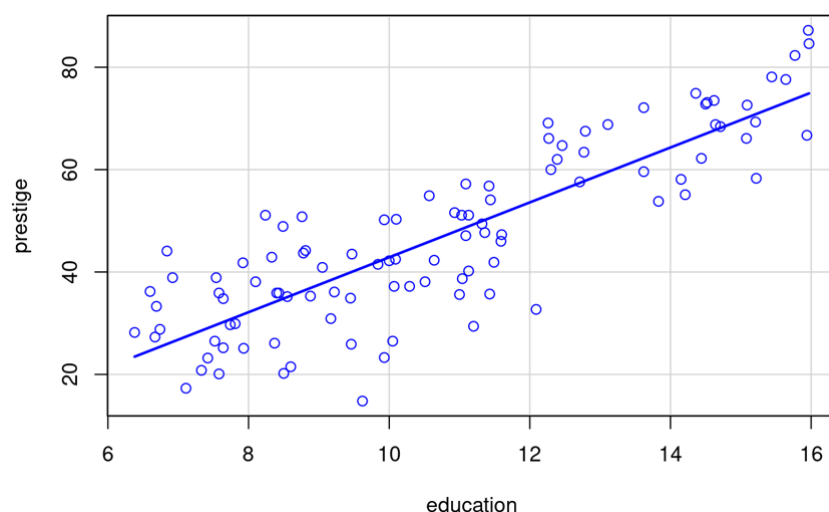


FIGURE 2 – Un graphique de la régression linéaire

somme des erreurs au carré entre les prédictions et les valeurs réelles.

- **Non Linéaire** : lorsque la relation nécessite une courbe plus complexe.
3. **Fonction de décision** : En régression, la fonction de décision modélise la relation entre les variables d'entrée (x) et la variable cible (y) pour prédire une valeur continue.
Par exemple, dans la régression linéaire, la fonction de décision prend la forme :
- $$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$
4. **Évaluation** : En régression, pour évaluer les performances du modèle, on peut utiliser des métriques telles que :

L'erreur Quadratique Moyenne Racine (Root Mean Square Error - RMSE) est utilisée pour mesurer l'écart entre les prédictions du modèle et les valeurs réelles.

- **RMSE est toujours non négatif** : $\text{RMSE} \geq 0$
- Plus le **RMSE** est faible, mieux les prédictions du modèle correspondent aux valeurs observées.
- Si **RMSE** = 0, cela signifie que le modèle est **parfait**.

Le coefficient de détermination (R^2), quant à lui, mesure la proportion de la variance des données expliquée par le modèle. Il permet d'évaluer la qualité d'ajustement du modèle. Il est calculé à l'aide des éléments suivants :

- **Somme des carrés résiduels (SS_{res})** : Mesure l'écart entre les valeurs réelles (y_i) et les valeurs prédites ($\hat{y}_i$).
- **Somme des carrés totaux (SS_{tot})** : Mesure l'écart entre les valeurs réelles (y_i) et leur moyenne ($\bar{y}$).

Formule :

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Avec :

$$SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{et} \quad SS_{\text{tot}} = \sum_{i=1}^n (y_i - \bar{y})^2$$

- Si $R^2 = 1$, le modèle est **parfait** : il explique entièrement la variance des données.

- Si $R^2 = 0$, le modèle n'explique **aucune** variance. Les prédictions sont équivalentes à utiliser simplement la moyenne des données.
- Si $0 < R^2 < 1$, le modèle explique une **proportion partielle** de la variance des données.
- Si $R^2 < 0$, le modèle est **moins performant** qu'une estimation basée sur la moyenne des données.

0.3.2 Processus de classification

La classification est utilisée pour prédire une catégorie ou une classe à laquelle appartient une observation, en fonction d'une ou plusieurs variables explicatives.

Étapes principales :

1. **Collecte des données** : Le dataset pour une tâche de classification contient des exemples où chaque entrée est associée à une étiquette de classe.
2. **Modélisation** : L'algorithme de classification apprend à partir des données d'entraînement pour établir une règle qui mappe les caractéristiques d'entrée aux étiquettes de classe.
3. **Fonction de décision** : L'algorithme apprend une fonction de décision qui sépare les différentes classes.

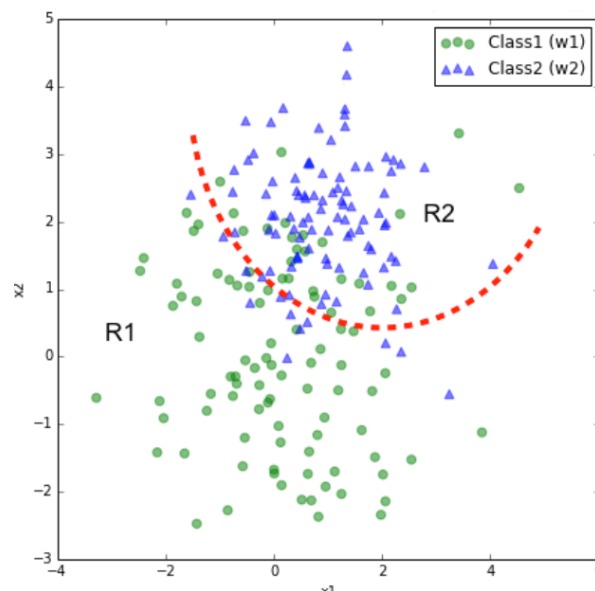


FIGURE 3 – Un graphique illustrant la bi-classification

Cette fonction peut être une frontière linéaire, ou une frontière plus complexe dans le cas d'algorithmes non linéaires (**Fig. 3**). L'objectif est de

trouver la meilleure séparation possible entre les différentes classes.

4. **Évaluation** : Le principe de l'évaluation d'un modèle de classification consiste à mesurer la capacité du modèle à prédire correctement les classes des données. On utilise un ensemble de données test, distinct de l'ensemble d'entraînement, et on compare les étiquettes prédites par le modèle avec les étiquettes réelles.

Pour cela, on se base sur divers méthodes :

- **Matrice de confusion** : une table qui résume le nombre de prédictions correctes et incorrectes.

	Prédictions Positives	Prédictions Négatives
Réel Positif	Vrai Positifs (VP)	Faux Négatifs (FN)
Réel Négatif	Faux Positifs (FP)	Vrai Négatifs (VN)

- **Précision (accuracy)** : la proportion de prédictions correctes parmi toutes les prédictions.

$$\text{Précision} = \frac{\text{Vrais Positifs (VP)}}{\text{Vrais Positifs (VP)} + \text{Faux Positifs (FP)}}$$

- **Courbe ROC** : un graphique qui trace le taux de vrais positifs (**TPR**) en fonction du taux de faux positifs (**FPR**) pour différents seuils de classification. Elle permet de visualiser la capacité d'un modèle à séparer les classes positives et négatives.

Formule :

$$\text{TPR} = \frac{\text{VP}}{\text{VP} + \text{FN}} \quad \text{et} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{VN}}$$

- **Courbe DET** : une courbe qui trace le taux de faux positifs (**FPR**) en fonction du taux de faux négatifs (**FNR**) pour différents seuils. Contrairement à la courbe ROC, la courbe DET met en évidence la relation entre les erreurs de classification.

Formule :

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{VP}}$$

Première Partie

Classification

0.4 Classification

0.4.1 Description de 5 modèles de classification

Dans cette section, nous nous concentrons sur cinq types de modèles de classification. Chaque modèle repose sur des approches mathématiques et algorithmiques spécifiques pour prédire les classes des données en fonction des caractéristiques fournies.

Nous allons examiner ces modèles en mettant en lumière leurs principes fondamentaux, leurs forces, et leurs limitations.

0.4.1.1 Arbre de Décision

Les arbres de décision sont des modèles utilisés en apprentissage automatique pour la classification ou la régression. Leur fonctionnement repose sur une structure arborescente où chaque nœud interne correspond à une condition sur une caractéristique, chaque branche représente un résultat de cette condition, et chaque feuille représente la classe ou la valeur prédite.

Les étapes du fonctionnement d'un arbre de décision sont les suivantes :

- **Sélectionner la caractéristique** qui sépare le mieux les données en fonction d'un critère comme l'entropie ou le gain d'information.
- **Séparer les données** en sous-groupes basés sur cette caractéristique.
- **Appliquer le même processus** aux sous-groupes obtenus, en répétant la division sur les caractéristiques restantes.
- **L'algorithme s'arrête** lorsqu'un critère d'arrêt est atteint, comme un nombre maximal de niveaux ou une pureté des groupes suffisante.

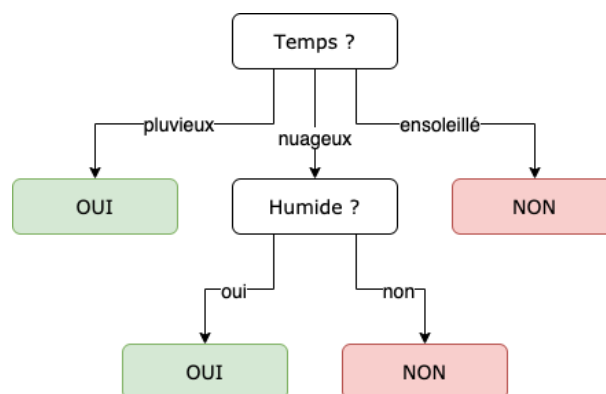


FIGURE 4 – Schéma illustrant un arbre de décision

Une fois l'arbre construit, il peut être utilisé pour prédire la classe ou la valeur d'une nouvelle donnée en suivant les conditions des nœuds jusqu'à une feuille.

Avantages : Les arbres de décision sont simples à comprendre, ne nécessitent pas de normalisation des données et peuvent gérer des relations non linéaires.

Inconvénients : Ils sont sensibles au surapprentissage, instables face aux variations des données et peuvent devenir trop complexes à interpréter.

0.4.1.2 Forêt Aléatoire

La Forêt Aléatoire (Random Forest) est un algorithme d'apprentissage supervisé qui combine plusieurs arbres de décision pour améliorer la précision des résultats.

Le fonctionnement du Random Forest suit les étapes suivantes :

- **Sélectionner aléatoirement** des sous-ensembles des données d'entraînement avec remplacement pour chaque arbre.
- **Pour chaque sous-ensemble**, construire un arbre de décision en sélectionnant aléatoirement un sous-ensemble de caractéristiques à chaque nœud, afin de garantir la diversité entre les arbres.
- **Vote** des arbres pour la classe la plus probable.
- **Agréger les résultats** de tous les arbres (vote majoritaire) pour donner la prédiction finale en utilisant la relation suivante :

$$\hat{y}_{\text{final}} = \arg \max_i \sum_{j=1}^N 1(\hat{y}_j = i)$$

où : $1(\hat{y}_j = i)$ est une fonction qui vaut 1 si la prédiction de l'arbre j est la classe i , et 0 sinon.

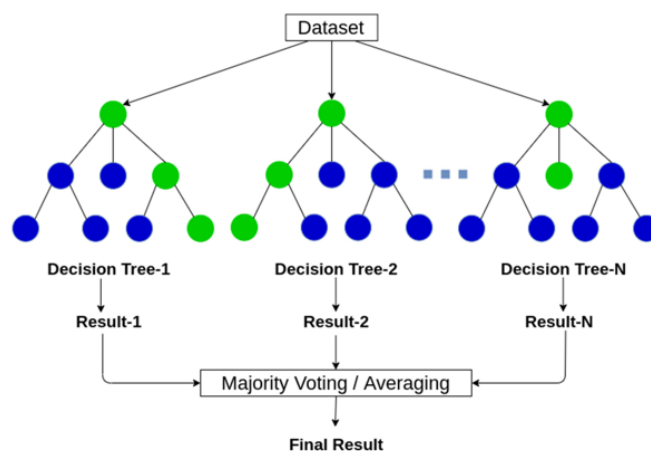


FIGURE 5 – Schéma illustrant une Forêt Aléatoire

Le **Random Forest** est robuste aux surapprentissage, efficace pour des problèmes de grande dimension et fournit une évaluation de l'importance des caractéristiques.

Par contre, il peut être lent à entraîner sur de grands ensembles de données, difficile à interpréter et nécessiter des ressources computationnelles importantes.

0.4.1.3 XGBoost

XGBoost (Extreme Gradient Boosting) est un modèle d'apprentissage automatique basé sur la technique de gradient boosting, qui combine plusieurs arbres de décision pour améliorer les prédictions. Contrairement au Random Forest qui combine des arbres de décision indépendants pour réduire la variance, XGBoost construit des arbres de manière séquentielle, de sorte que chaque nouvel arbre corrige les erreurs du précédent, ce qui permet de construire un modèle de plus en plus précis.

Les étapes clés du gradient boosting incluent :

- **Construction séquentielle des arbres** : Chaque arbre est construit pour corriger les erreurs (ou résidus) du modèle précédent.
- **Poids attribués aux erreurs** : Les erreurs sont pondérées pour que les erreurs les plus difficiles à corriger soient davantage prises en compte.
- **Optimisation par descente de gradient** : Le modèle minimise une fonction de perte en ajustant progressivement les prédictions de chaque arbre.

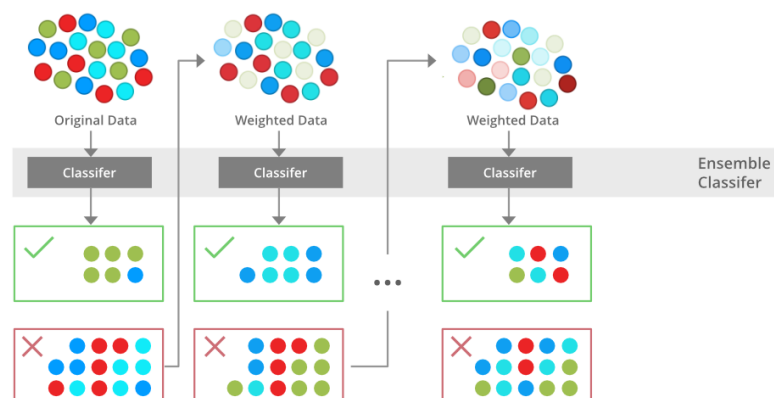


FIGURE 6 – Schéma illustrant le XGBoost

Le résultat final dans XGBoost est donnée alors par la somme pondérée des

prédictions de tous les arbres construits séquentiellement :

$$\hat{y}(x) = \sum_{t=1}^T \eta f_t(x)$$

où :

- $\hat{y}(x)$ est le résultat final de la prédiction pour l'entrée x ,
- T est le nombre total d'arbres,
- η est le taux d'apprentissage (learning rate),
- $f_t(x)$ est la prédiction de l'arbre t pour l'entrée x .

Avantages : XGBoost est rapide et efficace grâce à son optimisation en profondeur, il réduit le risque de surapprentissage grâce à la régularisation, et il gère bien les données manquantes et les caractéristiques non linéaires.

Inconvénients : Il peut être complexe à paramétrer correctement, il nécessite une grande quantité de mémoire pour de grands ensembles de données, et peut être sensible au bruit des données si le modèle est trop complexe.

0.4.1.4 SVM

La Machine à Vecteurs de Support (SVM) est un algorithme d'apprentissage supervisé utilisé pour des tâches de classification et de régression. L'idée principale de la méthode est de trouver l'hyperplan optimal qui sépare les données en différentes classes, en maximisant la marge entre les points les plus proches de chaque classe.

Les étapes du fonctionnement d'un SVM sont les suivantes :

- **Choisir un noyau** approprié (linéaire, polynôme, etc.) en fonction de la nature des données.
- **Calculer l'hyperplan optimal** qui sépare les classes en maximisant la marge entre les vecteurs de support.
- **Résoudre un problème d'optimisation** pour maximiser la marge tout en minimisant les erreurs de classification.
- **Prédiction :** Une fois l'hyperplan trouvé, il est utilisé pour classer de nouvelles données en fonction de leur position par rapport à l'hyperplan.

Un hyperplan est une frontière de décision qui sépare les points de données en différentes classes.

La marge est la distance entre l'hyperplan et les vecteurs de support. La SVM vise à maximiser cette marge, ce qui permet une meilleure généralisation du modèle.

Avantages : Les SVM sont efficaces pour des classes non linéaires, offrent une bonne capacité de généralisation et sont robustes aux dimensions élevées.

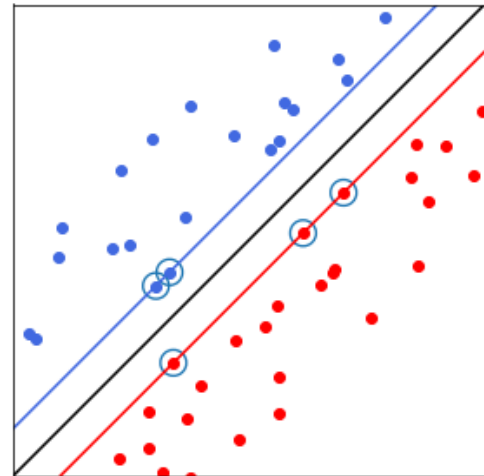


FIGURE 7 – Schéma illustrant le SVM

Inconvénients : Les SVM peuvent être sensibles aux paramètres de noyau et de régularisation, difficiles à interpréter pour des jeux de données complexes et peuvent être lents pour des ensembles de données très grands.

0.4.1.5 LogitBoost

Le modèle LogitBoost est un algorithme d'apprentissage supervisé qui combine les principes de la régression logistique et du boosting. Il est conçu pour résoudre des problèmes de classification binaire, et parfois multi-classes via des extensions.

Il fait partie de la famille des algorithmes de boosting, qui combine plusieurs modèles faibles pour former un modèle fort. L'idée principale est d'améliorer successivement la qualité des prédictions en pondérant les erreurs des échantillons mal classés à chaque étape, mais dans un cadre spécifique de logistique.

Étapes du fonctionnement de LogitBoost :

- **Initialisation :** Commencer avec un modèle de base, souvent une constante (par exemple, la classe majoritaire ou la probabilité initiale).
- **Calcul des résidus :** Calculer les résidus logistiques entre la prédiction actuelle et les vraies valeurs.
- **Construction des modèles faibles :** Construire un arbre de décision pour prédire les résidus en utilisant la fonction de perte logistique.
- **Mise à jour des prédictions :** Mettre à jour les prédictions en ajustant les erreurs précédentes, en ajoutant la prédiction du nouvel arbre avec un facteur d'apprentissage.

- **Répétition** : Répéter les étapes jusqu'à ce que le nombre d'arbres atteigne le maximum défini ou que la perte ne s'améliore plus.
- **Prédiction finale** : La prédiction finale est obtenue en combinant les prédictions des arbres construits.

La prédiction finale est obtenue en combinant les prédictions des arbres construits.

$$\hat{y}_{\text{final}} = \frac{1}{1 + \exp\left(-\sum_{t=1}^T \eta f_t(x)\right)}$$

où $f_t(x)$ représente les prédictions du modèle faible t , et η est le taux d'apprentissage.

LogitBoost est efficace pour les problèmes de classification binaire, gère bien les données déséquilibrées, et améliore la précision des prédictions grâce à son approche progressive d'ajustement des erreurs.

En revanche, il peut être sensible aux valeurs aberrantes, nécessite un temps d'entraînement plus long pour de grands ensembles de données, et peut souffrir de surapprentissage si le nombre d'arbres est trop élevé.

0.4.2 Le jeu de données de la maladie d'Alzheimer

L'analyse se concentre sur la prédiction de certaines maladies, en particulier la maladie d'**Alzheimer**. Cette maladie neurodégénérative affecte principalement la **mémoire**, mais touche également d'autres fonctions cognitives essentielles telles que le **langage**, le **raisonnement**, et l'**apprentissage**.

Pour cette étude, un jeu de données spécifique à la maladie d'**Alzheimer** a été utilisé, disponible sur **Kaggle**.

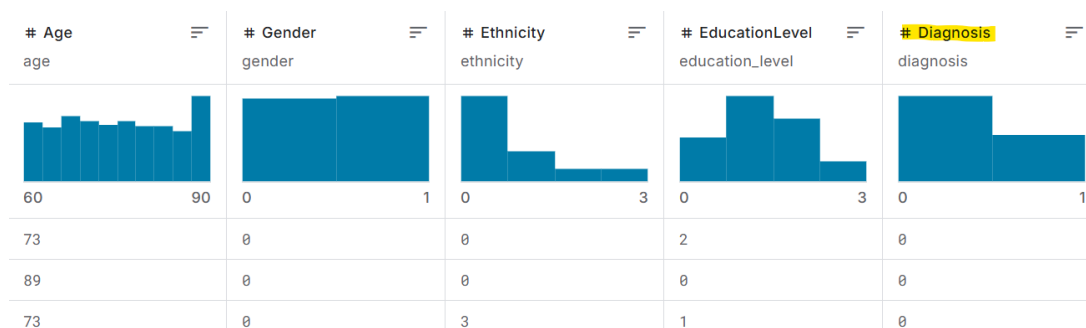


FIGURE 8 – Le jeu de données de la maladie d'Alzheimer

Ce jeu de données contient des informations complètes sur la santé de **2 149** patients, chacun identifié de manière unique par des numéros d'identification

allant de **4751 à 6900**.

Ces informations comprennent les détails indiqués ci-dessous :

Informations sur le Patient

Identifiant du patient

- **PatientID** : Identifiant unique attribué à chaque patient (allant de 4751 à 6900).

Données Démographiques

- **Age** : L'âge des patients varie de 60 à 90 ans.
- **Gender** : Sexe des patients, où 0 représente un homme et 1 une femme.
- **Ethnicity** : Origine ethnique des patients :
 - 0 : Caucasien
 - 1 : Afro-américain
 - 2 : Asiatique
 - 3 : Autre
- **EducationLevel** : Niveau d'éducation des patients :
 - 0 : Aucun
 - 1 : École secondaire
 - 2 : Baccalauréat
 - 3 : Plus élevé

Facteurs Liés au Mode de Vie

- **BMI** : Indice de masse corporelle des patients, compris entre 15 et 40.
- **Smoking** : Statut tabagique, où 0 indique Non et 1 indique Oui.
- **AlcoholConsumption** : Consommation hebdomadaire d'alcool en unités, comprise entre 0 et 20.
- **PhysicalActivity** : Activité physique hebdomadaire en heures, de 0 à 10.
- **DietQuality** : Score de qualité du régime alimentaire, allant de 0 à 10.
- **SleepQuality** : Score de qualité du sommeil, allant de 4 à 10.

Antécédents Médicaux

- **FamilyHistoryAlzheimers** : Antécédents familiaux de maladie d'Alzheimer, où 0 indique Non et 1 indique Oui.

- **CardiovascularDisease** : Présence d'une maladie cardiovasculaire, 0 indiquant Non et 1 indiquant Oui.
- **Diabetes** : Présence de diabète, où 0 indique Non et 1 indique Oui.
- **Depression** : Présence d'une dépression, où 0 indique Non et 1 indique Oui.
- **HeadInjury** : Antécédents de traumatisme crânien, où 0 indique Non et 1 indique Oui.
- **Hypertension** : Présence d'hypertension, où 0 indique Non et 1 indique Oui.

Mesures Cliniques

- **SystolicBP** : Tension artérielle systolique, comprise entre 90 et 180 mmHg.
- **DiastolicBP** : Tension artérielle diastolique, comprise entre 60 et 120 mmHg.
- **CholesterolTotal** : Taux de cholestérol total, compris entre 150 et 300 mg/dL.
- **CholesterolLDL** : Taux de cholestérol des lipoprotéines de basse densité, compris entre 50 et 200 mg/dL.
- **CholesterolHDL** : Taux de cholestérol des lipoprotéines de haute densité, compris entre 20 et 100 mg/dL.
- **CholesterolTriglycerides** : Taux de triglycérides, compris entre 50 et 400 mg/dL.

Évaluations Cognitives et Fonctionnelles

- **MMSE** : Score du Mini-Mental State Examination, compris entre 0 et 30. Les scores les plus bas indiquent une déficience cognitive.
- **FunctionalAssessment** : Score d'évaluation fonctionnelle, allant de 0 à 10. Les scores les plus bas indiquent une déficience plus importante.
- **MemoryComplaints** : Présence de troubles de la mémoire, où 0 indique Non et 1 indique Oui.
- **BehavioralProblems** : Présence de problèmes de comportement, où 0 indique Non et 1 indique Oui.
- **ADL** : Score des activités de la vie quotidienne, allant de 0 à 10. Les scores les plus bas indiquent une déficience plus importante.

Symptômes

- **Confusion** : Présence de confusion, où 0 indique Non et 1 indique Oui.
- **Disorientation** : Présence de désorientation, où 0 indique Non et 1 indique Oui.
- **PersonalityChanges** : Présence de changements de personnalité, où 0 indique Non et 1 indique Oui.
- **DifficultyCompletingTasks** : Présence de difficultés à accomplir des tâches, où 0 indique Non et 1 indique Oui.
- **Forgetfulness** : Présence d'oublis, où 0 indique Non et 1 indique Oui.

Informations sur le Diagnostic

- **Diagnostic** : Statut du diagnostic de la maladie d'Alzheimer, où 0 indique Non et 1 indique Oui.

Informations Confidentielles

- **DoctorInCharge** : Cette colonne contient des informations confidentielles sur le médecin responsable, avec « XXXConfid » comme valeur pour tous les patients.

Les données contiennent un total de 35 colonnes, dont 34 représentent des caractéristiques (*features*) décrivant les patients et 1 colonne est dédiée au statut du diagnostic (**Diagnostic**).

0.4.3 Prétraitement des données

Le prétraitement des données est une étape essentielle pour assurer la qualité et la pertinence des analyses. Dans cette phase, plusieurs vérifications ont été effectuées pour évaluer l'état initial des données et préparer leur utilisation.

Tout d'abord, les données ont été **chargées** et un aperçu initial a été effectué pour évaluer leur structure. Le jeu de données se compose principalement de **variables** numériques, et une vérification a permis de confirmer qu'il n'y avait **pas de valeurs manquantes**.

```
> data <- read.csv("alzheimers_disease_data.csv")
> str(data)
> colSums(is.na(data))
```

Cependant, il a été observé que certaines colonnes du jeu de données, telles que l'**ID** et les deux variables "**DoctorInCharge**", ne sont pas significatives pour les analyses à venir. Ces colonnes ont donc été supprimées, car elles ne contribuaient pas à l'information utile pour la modélisation.

```
> subset(data, select=-c(DoctorInCharge, PatientID))
```

Une fois ces étapes de nettoyage effectuées, nous avons appliqué deux méthodes de réduction de dimensionnalité pour mieux comprendre les données et faciliter leur traitement.

0.4.3.1 Analyse de la Corrélation

Pour simplifier le modèle et se concentrer sur les variables les plus pertinentes, une analyse de **corrélation** a été effectuée sur le jeu de données. L'objectif principal était de comprendre les relations linéaires entre les variables, en particulier par rapport à la variable cible, **Diagnosis**, qui indique si un patient présente la maladie d'**Alzheimer**.

0.4.3.2 Heatmap de la matrice de corrélation

Une heatmap a été générée pour visualiser la matrice de corrélation entre toutes les variables.

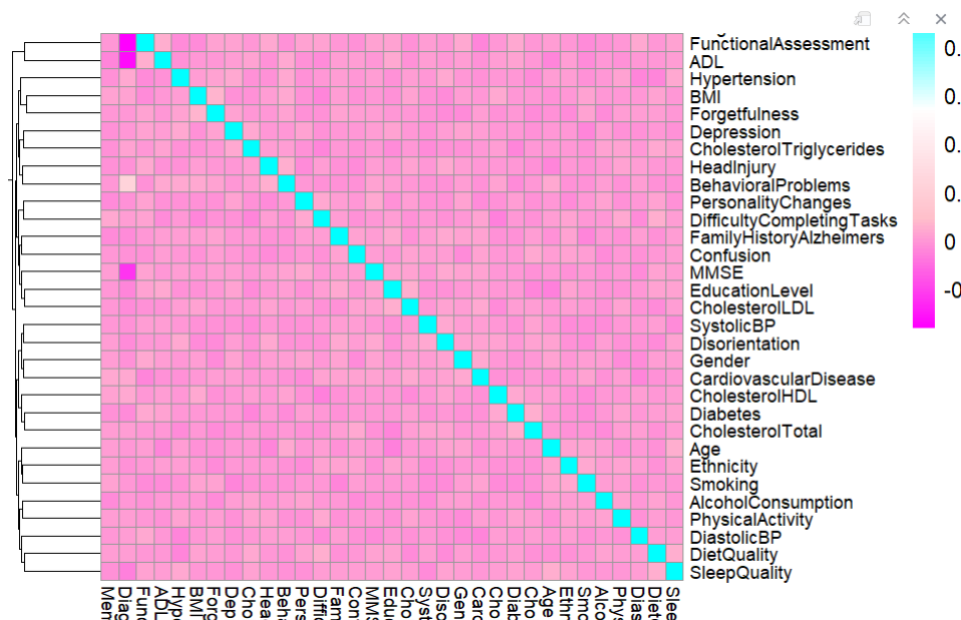


FIGURE 9 – Heatmap du jeu de données

Les couleurs indiquent la force et la direction de la corrélation :

- **Cyan** : Corrélations positives fortes, proches de 1.
- **Magenta** : Corrélations négatives fortes, proches de -1.

- **Entre Cyan et Magenta** : Corrélations intermédiaires.

La heatmap est particulièrement utile pour observer les relations entre les variables, mais elle peut devenir difficile à interpréter lorsque le nombre de variables est élevé.

Dans ce cas (**Fig.5**), il est préférable de se concentrer sur la corrélation des variables avec la variable cible, **Diagnosis**, afin de mieux comprendre quelles caractéristiques influencent la prédiction de la maladie d'Alzheimer.

0.4.3.3 Calcul des corrélations avec la variable cible (**Diagnosis**) :

L'analyse s'est focalisée sur l'extraction des **corrélations** entre chaque variable et **Diagnosis**. Cela permet d'identifier les variables qui sont les plus fortement corrélées à la maladie d'Alzheimer et qui, par conséquent, pourraient être les plus importantes pour le modèle.

```
> cor_matrix <- cor(data[,sapply(data, is.numeric)])
> cor_with_diagnosis <- cor_matrix[ , "Diagnosis"]
> cor_with_diagnosis
```

Cette étape de sélection des variables repose sur un seuil prédéfini pour la corrélation absolue. Par exemple, seules les variables ayant une corrélation absolue avec **Diagnosis** supérieure à **0.02** ont été conservées. Cela réduit la dimensionnalité tout en préservant les informations essentielles pour la prédiction.

```
> seuil <- 0.02
> cor_diagno <- cor(data_numeric)["Diagnosis", ]
> cor_abs <- abs(cor_diagno)
> selected_features <- names(cor_diagno[cor_abs > seuil])
```

0.4.3.4 Impact de la sélection des variables

Avant le filtrage, le jeu de données comprenait **33 variables**.

[1] "Gender"	"EducationLevel"
[3] "BMI"	"SleepQuality"
[5] "FamilyHistoryAlzheimers"	"CardiovascularDisease"
[7] "Diabetes"	"HeadInjury"
[9] "Hypertension"	"CholesterolLDL"
[11] "CholesterolHDL"	"CholesterolTriglycerides"
[13] "MMSE"	"FunctionalAssessment"
[15] "MemoryComplaints"	"BehavioralProblems"
[17] "ADL"	"Disorientation"
[19] "PersonalityChanges"	"Diagnosis"

FIGURE 10 – Variables sélectionnées pour l'entraînement

Après le filtrage, seules les caractéristiques significatives ont été retenues, **au total de 19**, ce qui simplifie le modèle et améliore potentiellement sa performance. En se concentrant sur les variables les plus corrélées avec **Diagnosis**, nous réduisons également le risque de surapprentissage et optimisons l'interprétabilité du modèle.

0.4.3.5 Analyse en composantes principales

L'Analyse en composantes principales (ACP) est une méthode statistique utilisée pour réduire la dimensionnalité d'un jeu de données tout en préservant autant que possible la variabilité des données.

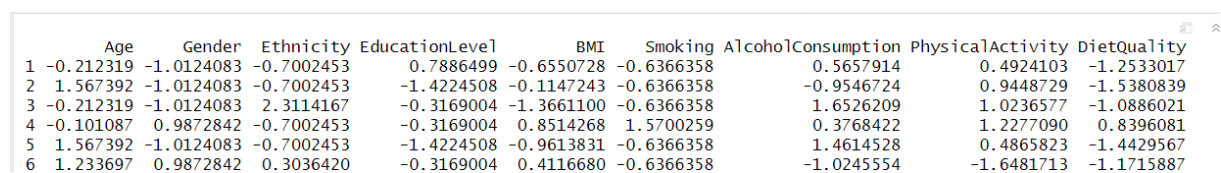
Étant donné le grand nombre de colonnes, ce qui complique le traitement des données, l'usage de l'Analyse en Composantes Principales (ACP) constitue également une méthode appropriée pour réduire la dimensionnalité et simplifier l'analyse.

0.4.3.6 Normalisation

On commence par supprimer la colonne **Diagnosis** puisqu'elle est celle de sortie, puis on normalise nos données (centrage et réduction) afin d'égaliser les échelles des variables.

Cela garantit que les attributs contribuent équitablement à l'analyse en composantes principales, évitant qu'une variable à forte variance domine les résultats.

```
> data_ <- subset(data, select = -Diagnosis)
> data_scaled <- scale(data_)
```



	Age	Gender	Ethnicity	EducationLevel	BMI	Smoking	AlcoholConsumption	PhysicalActivity	DietQuality
1	-0.212319	-1.0124083	-0.7002453	0.7886499	-0.6550728	-0.6366358	0.5657914	0.4924103	-1.2533017
2	1.567392	-1.0124083	-0.7002453	-1.4224508	-0.1147243	-0.6366358	-0.9546724	0.9448729	-1.5380839
3	-0.212319	-1.0124083	2.3114167	-0.3169004	-1.3661100	-0.6366358	1.6526209	1.0236577	-1.0886021
4	-0.101087	0.9872842	-0.7002453	-0.3169004	0.8514268	1.5700259	0.3768422	1.2277090	0.8396081
5	1.567392	-1.0124083	-0.7002453	-1.4224508	-0.9613831	-0.6366358	1.4614528	0.4865823	-1.4429567
6	1.233697	0.9872842	0.3036420	-0.3169004	0.4116680	-0.6366358	-1.0245554	-1.6481713	-1.1715887

FIGURE 11 – Tableau des données normalisées

0.4.3.7 Calcul des Valeurs Propres

On applique alors la fonction 'prcomp' aux données normalisées qui retourne un objet contenant plusieurs éléments utiles pour interpréter le résultat de l'ACP.

```
> pca_result <- prcomp(data_scaled, center=F, scale.=F)
```

La sortie `pca_result$sdev` contient les écarts-types associés aux composantes principales. En élevant les écarts-types au carré, on obtient les valeurs propres qui correspondent à la quantité de variance expliquée par chaque composante.

```
> eigenvalues <- pca_result$sdev^2

[1] 1.2103626 1.1976593 1.1816109 1.1733275 1.1367974 1.1255545 1.1073189 1.0965018 1.0875536 1.0787328 1.0543078
[12] 1.0425540 1.0354343 1.0170945 1.0154299 0.9960425 0.9839300 0.9689604 0.9604972 0.9567139 0.9496144 0.9458545
[23] 0.9227220 0.9140001 0.9020990 0.8912102 0.8717314 0.8589152 0.8530502 0.8386388 0.8236066 0.8021737
```

FIGURE 12 – Tableau des Valeurs Propres

0.4.3.8 Détermination de la proportion de la variance totale expliquée

La variance cumulée est calculée pour déterminer quelle proportion de la variance totale est expliquée en ajoutant successivement chaque composante principale.

Nous calculons la somme cumulative des valeurs propres, et nous divisons par la somme totale des valeurs propres pour normaliser les valeurs.

```
> cum_var <- cumsum(eigenvalues) / sum(eigenvalues)
> num_pcs <- which(cum_var >= 0.85)[1]
> data_pca <- as.data.frame(pca_result$x)
> data_reduced <- data_pca[, 1:num_pcs]
```

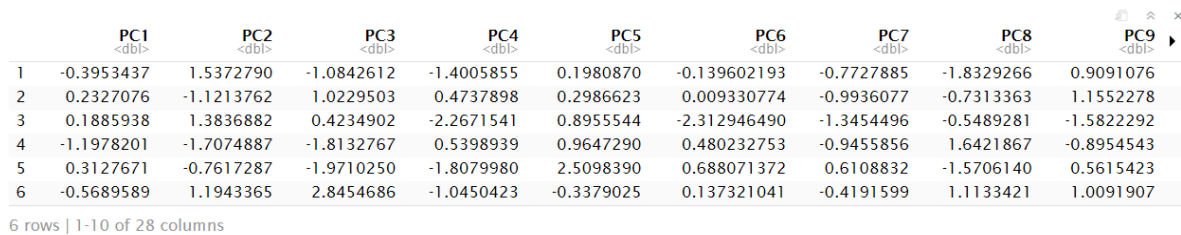
Par la suite, nous sélectionnons le nombre optimal de composantes principales de manière à conserver un pourcentage suffisant de la variance totale des données.

Pour ce faire, nous utilisons la somme cumulative des valeurs propres afin d'identifier le nombre minimal de composantes principales nécessaires. Ensuite, nous extrayons les scores des composantes principales à partir des résultats de l'ACP et conservons uniquement les composantes sélectionnées dans une matrice réduite.

Choix du pourcentage de la variable totale des données :

Pourcentage de Variable Totale	Nombre de Colonnes	Accuracy d'entraînement	Accuracy de Testing
95%	32	89.94%	83.68%
90%	30	86.8%	80.89%
85%	28	88.66%	84.38%
80%	26	87.85%	79.25%
75%	24	85.23%	80.19%

Il en ressort que le modèle offrant les meilleures performances (accuracy) est celui qui utilise les composantes principales représentant au moins **85 % de la variance** totale des données. Ce pourcentage est donc sélectionné.



	PC1 <dbl>	PC2 <dbl>	PC3 <dbl>	PC4 <dbl>	PC5 <dbl>	PC6 <dbl>	PC7 <dbl>	PC8 <dbl>	PC9 <dbl>
1	-0.3953437	1.5372790	-1.0842612	-1.4005855	0.1980870	-0.139602193	-0.7727885	-1.8329266	0.9091076
2	0.2327076	-1.1213762	1.0229503	0.4737898	0.2986623	0.009330774	-0.9936077	-0.7313363	1.1552278
3	0.1885938	1.3836882	0.4234902	-2.2671541	0.8955544	-2.312946490	-1.3454496	-0.5489281	-1.5822292
4	-1.1978201	-1.7074887	-1.8132767	0.5398939	0.9647290	0.480232753	-0.9455856	1.6421867	-0.8954543
5	0.3127671	-0.7617287	-1.9710250	-1.8079980	2.5098390	0.688071372	0.6108832	-1.5706140	0.5615423
6	-0.5689589	1.1943365	2.8454686	-1.0450423	-0.3379025	0.137321041	-0.4191599	1.1133421	1.0091907

6 rows | 1-10 of 28 columns

FIGURE 13 – Tableau des PCAs

Nous avons alors moins de colonnes avec des données plus significatives par rapport au diagnostic.

0.4.4 Entraînement et Comparaison entre les modèles

Nous avons testé les **5 modèles** de classifications choisis sur le dataset de la maladie d'**Alzheimer** avec des méthodes de prétraitements des données différentes en l'occurrence l'**Analyse en composantes principales (ACP)** et **la corrélation**, ce qui nous a donné le tableau ci-dessous :

Types de Modèles	Méthode de pré-traitement	Accuracy Training	Accuracy Test
Arbre de décision	ACP	70.81%	72.02%
SVM	ACP	88.66%	84.38%
Random forest	—	90.29%	90%
XGBoost	Corrélation	94.08%	94.25%
LogitBoot	Corrélation	95.27%	94.41%

TABLE 1 – Performance des modèles en fonction des méthodes de prétraitement

À partir du tableau précédent, nous allons maintenant examiner en détail l'implémentation de deux modèles utilisant des méthodes distinctes pour le prétraitement des données, comme décrit précédemment.

Les modèles sélectionnés sont la machine à vecteurs de support (**SVM**) combinée à l'analyse en composantes principales (ACP), ainsi qu'**XGBoost** utilisant la corrélation comme méthode de prétraitement.

0.4.4.1 Partition des données en ensembles d'entraînement et de test

La première étape de l'entraînement d'un modèle consiste à diviser le jeu de données en deux sous-ensembles : **l'ensemble d'entraînement** et **l'ensemble de test**.

Cela permet de former le modèle sur un sous-ensemble des données (entraînement) et d'évaluer ses performances sur un autre sous-ensemble (test), assurant ainsi une estimation objective de sa capacité à généraliser à de nouvelles données.

Pour ce faire, nous utilisons la fonction **createDataPartition** du package **caret**. Cette fonction permet de créer une partition aléatoire des données, tout en garantissant que la distribution des classes dans les ensembles d'entraînement et de test soit similaire à celle du jeu de données d'origine.

```
> set.seed(123)
> diag <- data$Diagnosis
> train_index <- createDataPartition(diag, p=0.8, list=F)

> train_data <- data[train_index, ]
> test_data <- data[-train_index, ]

> X_train <- train_data[, -ncol(train_data)]
> Y_train <- train_data$Diagnosis
> X_test <- test_data[, -ncol(test_data)]
> Y_test <- test_data$Diagnosis
```

Dans notre cas, **80 %** des données sont utilisées pour l'entraînement et les **20 %** restants sont affectés à l'ensemble de test.

0.4.5 Entraînement du modèle SVM

Une fois les données partitionnées, nous pouvons procéder à l'entraînement du modèle SVM. Le modèle SVM (Support Vector Machine) est un algorithme de classification puissant, notamment lorsqu'il est utilisé avec des noyaux non linéaires. Nous utilisons ici un noyau radial (**svmRadial**) qui est couramment utilisé pour ses bonnes performances dans des problèmes complexes.

```
> svm_model <- train(
  x = x_train,
  y = y_train,
  method = "svmRadial",
  trControl = trainControl(method = "cv", number = 5)
)
```

- **x = x_train** : Ce paramètre désigne le jeu de données d'entraînement contenant les caractéristiques.
- **y = y_train** : Ce paramètre correspond au vecteur des étiquettes de la variable cible.
- **method = "svmRadial"** : Ce paramètre spécifie l'algorithme utilisé pour l'entraînement du modèle. Ici, il désigne un SVM avec un noyau radial (RBF).

Ce noyau mesure la similarité entre deux points de données x et x' en utilisant la fonction gaussienne suivante :

$$K(x, x') = \exp \left(-\frac{\|x - x'\|^2}{2\sigma^2} \right)$$

Où :

- $\|x - x'\|$ est la distance euclidienne entre les points x et x' ,
 - σ est un paramètre qui contrôle la largeur de la fonction gaussienne.
- **trControl = trainControl(method = "cv", number = 5)** : Ce paramètre définit la méthode utilisée pour évaluer le modèle pendant l'entraînement.
 - **method = "cv"** : Cela spécifie que la validation croisée (cross-validation) sera utilisée pour évaluer la performance du modèle. La validation croisée consiste à diviser les données en plusieurs sous-ensembles (plis), à entraîner le modèle sur certains de ces sous-ensembles et à évaluer ses performances sur les autres.
 - **number = 5** : Ce paramètre indique que la validation croisée sera effectuée avec 5 plis. Le modèle sera entraîné et testé 5 fois, chaque fois avec un pli différent comme ensemble de test et les autres plis comme ensemble d'entraînement.

0.4.6 Entraînement du modèle XGBoost

La fonction `train()` du package `caret` est utilisée pour entraîner le modèle XGBoost. Nous utilisons la validation croisée (CV) pour évaluer la performance du modèle et une **grille d'hyperparamètres** (`tuneGrid`) pour optimiser les paramètres du modèle.

```
> tuneGrid <- expand.grid(
  nrounds = seq(10, 100, by = 10),
  max_depth = c(6, 8, 10),
  eta = c(0.01, 0.1, 0.2),
  gamma = c(0, 0.1, 0.2),
  colsample_bytree = c(0.7, 0.8, 0.9),
  min_child_weight = c(1, 5, 10),
  subsample = c(0.7, 0.8, 0.9)
)
```

Grille d’hyperparamètres (tuneGrid) : La grille d’hyperparamètres contient des valeurs spécifiques pour chaque paramètre que le modèle XGBoost va tester lors de l’entraînement :

- **nrounds** : Le nombre d’itérations de boosting, ou le nombre d’arbres à construire.
- **max_depth** : La profondeur maximale des arbres.
- **eta** : Le taux d’apprentissage, qui détermine l’ampleur des ajustements effectués par le modèle à chaque itération. Des valeurs faibles permettent une convergence plus fine, mais nécessitent plus d’itérations.
- **gamma** : Paramètre de régularisation qui contrôle la complexité des arbres.
- **colsample_bytree** : La proportion des caractéristiques utilisées pour chaque arbre.
- **min_child_weight** : Le poids minimal des feuilles d’arbre. Ce paramètre influence la taille des sous-arbres : des valeurs plus élevées rendent le modèle plus conservateur et évitent la division des sous-arbres qui n’ont pas suffisamment d’exemples.
- **subsample** : La fraction d’échantillons utilisés pour entraîner chaque arbre.

Après avoir défini la grille des hyperparamètres, nous passons à la configuration de la validation croisée et à l’entraînement du modèle.

```
> control <- trainControl(method = "cv",
  number = 5, verboseIter = TRUE)
> model_xgb <- train(
  Diagnosis ~ .,
  data = train_data,
```

```
method = "xgbTree",  
trControl = control,  
tuneGrid = tuneGrid,  
metric = "Accuracy" )
```

trainControl(method = "cv", number = 5) : La fonction **trainControl** permet de définir la méthode de validation croisée (cross-validation) utilisée pour évaluer le modèle.

- **method = "cv"** : Ce paramètre spécifie la méthode de validation croisée utilisée.
- **number = 5** : Ce paramètre définit le nombre de plis à utiliser pour la validation croisée. Dans notre cas, un total de 5 plis est utilisé, ce qui signifie que les données seront divisées en 5 sous-ensembles, et chaque sous-ensemble servira à la fois de jeu d'entraînement et de jeu de test au cours de l'entraînement du modèle.

train(Diagnosis ~ ., data = train_data, method = "xgbTree", trControl = train_control, tuneGrid = tuneGrid) : La fonction **train** est utilisée pour entraîner le modèle en utilisant les données d'entraînement et la configuration spécifiée par la fonction **trainControl**.

- **Diagnosis ~ .** : Cette formule indique que le modèle doit prédire la variable cible **Diagnosis** en fonction de toutes les autres variables du jeu de données d'entraînement.
- **data = train_data** : Ce paramètre spécifie le jeu de données utilisé pour l'entraînement du modèle.
- **method = "xgbTree"** : Ce paramètre spécifie le modèle d'apprentissage à utiliser. "xgbTree" indique que l'on souhaite entraîner un modèle basé sur XGBoost (Extreme Gradient Boosting) avec des arbres de décision.
- **trControl = train_control** : Ce paramètre définit l'objet **trainControl** qui contient les paramètres de validation croisée.
- **tuneGrid = tuneGrid** : Ce paramètre spécifie la grille d'hyperparamètres (**tuneGrid**) que le modèle doit tester pour trouver la meilleure combinaison de paramètres.

0.4.7 Évaluation des Modèles

L'évaluation des modèles constitue une étape cruciale pour analyser leurs performances. Dans cette section, nous nous concentrerons d'abord sur l'analyse de la courbe de précision au cours de l'entraînement, afin de comprendre l'évolution des performances du modèle. Ensuite, nous examinerons la courbe ROC et la courbe DET, qui offrent des perspectives complémentaires sur les compromis entre les erreurs et les performances globales.

0.4.7.1 Évaluation du Modèle SVM

C est un paramètre important dans les modèles SVM qui contrôle la pénalité pour les erreurs de classification. Un C faible permet au modèle de tolérer plus d'erreurs pour une meilleure généralisation, tandis qu'un C élevé cherche à minimiser les erreurs de classification.

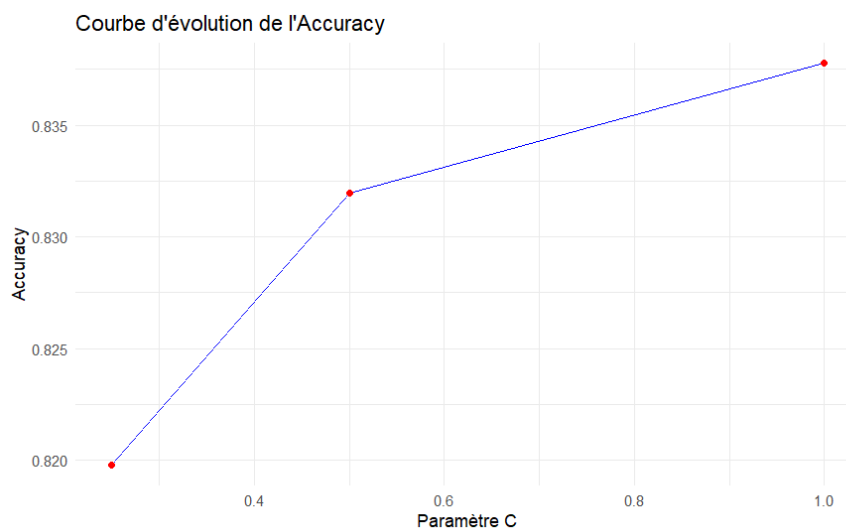


FIGURE 14 – Courbe illustrant l'évolution de la précision par rapport à C .

La courbe montre une augmentation progressive de l'accuracy avec l'augmentation du paramètre C , ce qui montre que, dans ce cas, une pénalité plus forte pour les erreurs améliore la performance du modèle. Cela indique que le modèle bénéficie d'une moins grande tolérance aux erreurs, le rendant plus précis dans la classification des données d'entraînement.

$C = 1$ offre une meilleure performance (accuracy de 84.38%), indiquant que ce paramètre est proche de son optima pour ce problème particulier. Le modèle trouve un bon compromis entre la capacité à classer les données sans faire trop d'erreurs tout en restant suffisamment général pour éviter le sur-apprentissage.

0.4.7.2 Évaluation des métriques de performance

Dans cette analyse, nous avons évalué la performance de notre modèle de classification en utilisant la courbe ROC.

Les résultats obtenus sur les données de test indiquent que le modèle a correctement prédit environ 84,38% des échantillons. Cela montre que le modèle a une capacité raisonnable à généraliser et à fournir des prédictions fiables sur de nouvelles données, bien qu'il existe encore une marge d'amélioration.

1. Courbe ROC :

La courbe ROC s'élève rapidement dans les premiers segments, indiquant que le modèle atteint un fort taux de vrais positifs (TPR) tout en gardant le taux de faux positifs (FPR) faible.

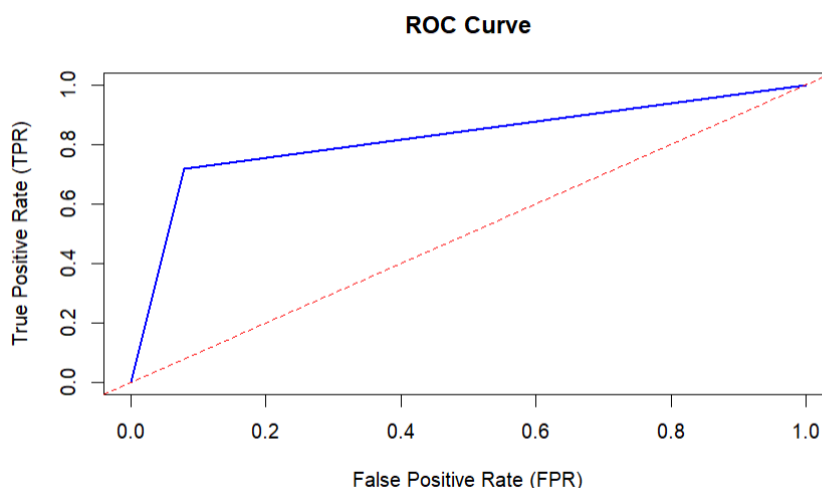


FIGURE 15 – Courbe ROC du modèle SVM

À mesure que le FPR augmente, l'amélioration du TPR devient moins marquée, ce qui est courant. Cela reflète une phase où l'ajout de plus de prédictions positives entraîne également plus de faux positifs.

Cette courbe ROC suggère que le modèle a une bonne performance, particulièrement dans les zones critiques où la réduction des faux positifs est importante.

2. Courbe DET :

Une courbe proche de l'origine (en bas à gauche) indique un bon équilibre entre FPR et FNR. Ici, la courbe montre une bonne performance globale, surtout pour des seuils où le compromis est modéré (FPR faible

à moyen).

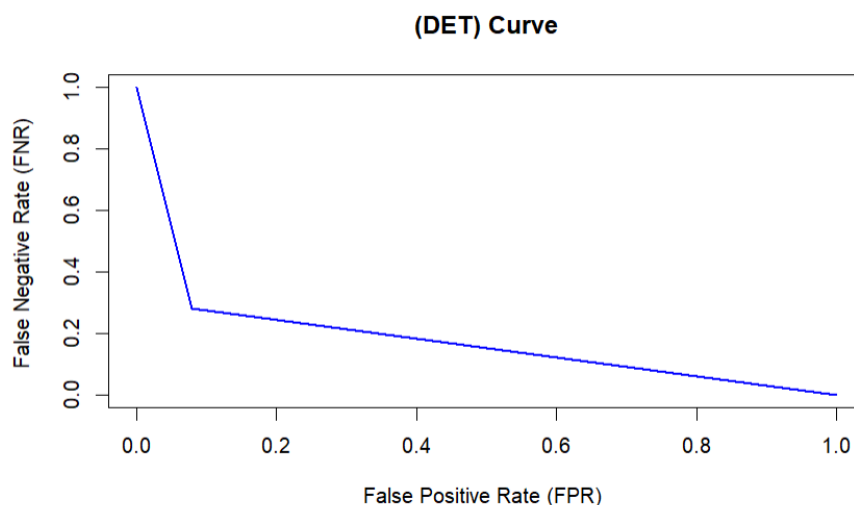


FIGURE 16 – Courbe DET du modèle SVM

La courbe commence en haut à gauche, où le FNR est élevé (près de 1) mais le FPR est faible (près de 0). Cela signifie que le modèle rejette presque toutes les observations comme négatives, entraînant de nombreux faux négatifs.

La courbe devient presque plate lorsque le FPR approche 1. Cela signifie qu'à ce stade, toutes les observations sont classées comme positives, entraînant un FNR faible mais un FPR très élevé.

La courbe DET confirme que le modèle est bien calibré et offre un compromis satisfaisant entre les erreurs de type faux positifs et faux négatifs. Cependant, le choix final du seuil doit être guidé par les priorités spécifiques du problème : minimiser les faux positifs, les faux négatifs ou trouver un équilibre.

0.4.7.3 Évaluation du Modèle XGBoost

La courbe montre l'évolution de la précision au cours de l'entraînement du modèle, où l'accuracy est tracée en fonction du nombre d'itérations .

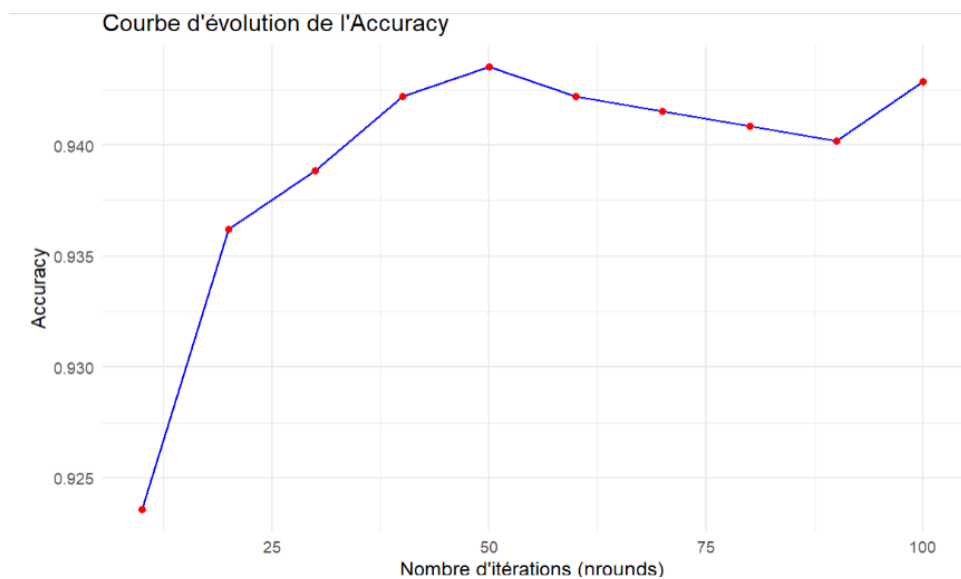


FIGURE 17 – Courbe illustrant l'évolution de la précision au cours de l'entraînement

Initialement : Nous observons une augmentation rapide de la précision au début de l'entraînement. Dans cette phase, la précision grimpe de 0.925 à environ 0.935 entre les premières itérations.

Période intermédiaire : Ensuite, la courbe connaît une augmentation plus progressive, atteignant un pic autour de l'itération 50, où la précision semble se stabiliser autour de 0.940. Cela montre que le modèle continue d'améliorer ses performances, mais à un rythme moins rapide.

Stabilisation et plateau : Vers la fin, après l'itération 70, la précision reste relativement stable ou subit de petites fluctuations autour de 0.940, ce qui signifie que le modèle a presque convergé et que les améliorations deviennent marginales. Cela indique que l'apprentissage est arrivé à un plateau.

0.4.7.4 Évaluation des métriques de performance

La performance obtenue sur les données de test indique que le modèle a correctement prédit environ 94.25% des échantillons dans l'ensemble de test. Cela signifie que le modèle est capable de généraliser efficacement sur des données non vues.

1. Courbe ROC :

La courbe ROC s'élève rapidement vers le coin supérieur gauche, ce qui indique que le modèle a un faible FPR tout en maintenant un TPR élevé.

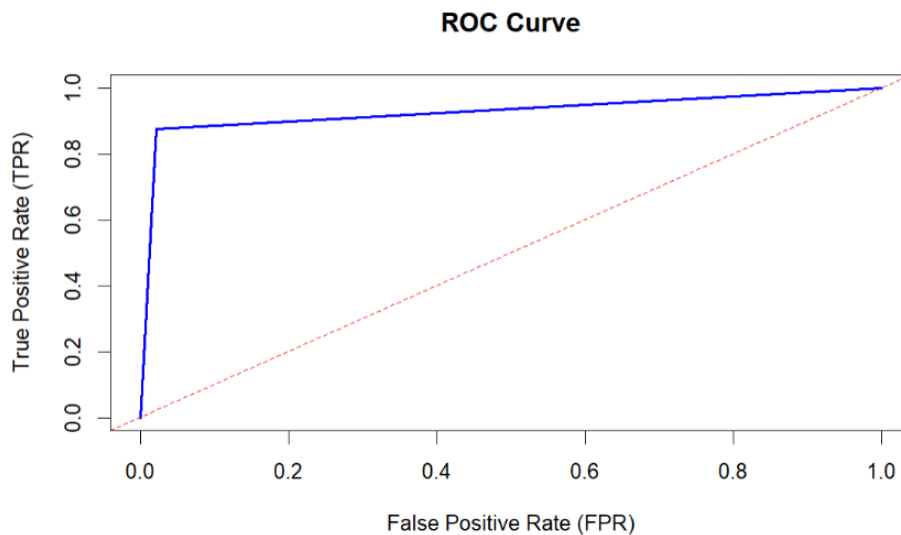


FIGURE 18 – Courbe ROC du modèle XGBoost

Cela signifie que le modèle commet peu d'erreurs en classant à tort des exemples négatifs comme positifs. En d'autres termes, il maintient un faible taux de faux positifs, ce qui montre sa bonne capacité prédictive.

2. Courbe DET :

La courbe DET est généralement une fonction décroissante : lorsqu'une erreur (comme les faux positifs) diminue, l'autre erreur (comme les faux négatifs) peut augmenter.

Ici, la courbe descend rapidement et tend vers le bas, ce qui montre que le modèle réussit à réduire simultanément les deux types d'erreurs dans une large mesure.

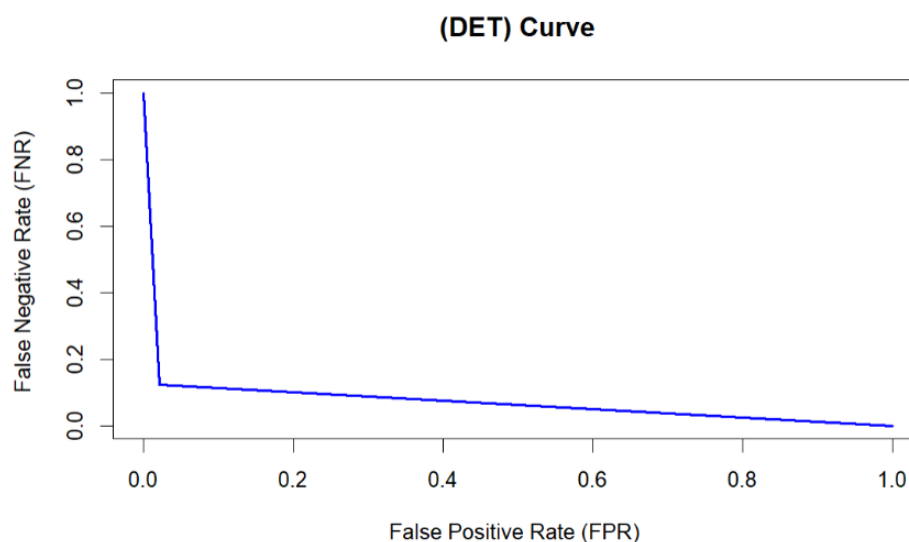


FIGURE 19 – Courbe DET du modèle XGBoost

La forme de la courbe DET indique que le modèle maintient un équilibre optimal entre le taux de faux positifs et le taux de faux négatifs. Cela signifie que le modèle est efficace, car il minimise les deux types d'erreurs sans trop de compromis.

0.4.8 Conclusion

Les modèles **SVM** et **XGBoost** présentent des performances notables dans la classification, mais leurs approches diffèrent sensiblement.

Le modèle SVM avec **ACP** a montré une accuracy de **84,38%**, ce qui démontre une capacité décente à prédire les classes tout en maintenant un équilibre entre la minimisation des erreurs et la généralisation. La courbe ROC de ce modèle met en évidence un bon taux de vrais positifs avec un faible taux de faux positifs, et la courbe DET montre un compromis équilibré entre les erreurs de classification.

D'un autre côté, le modèle XGBoost, utilisant la méthode de la **corrélation**, a montré une performance encore plus impressionnante avec une accuracy de **94,25%**. Ce modèle a su généraliser efficacement sur les nouvelles données, en maintenant un faible taux de faux positifs et un haut taux de vrais positifs selon la courbe ROC. De plus, la courbe DET de XGBoost indique un équilibre optimal entre les erreurs, avec une réduction significative à la fois des faux positifs et des faux négatifs.

Deuxième Partie

Régression

0.5 Régression

0.5.1 Description de 5 modèles de régression

Dans cette section, nous explorons **cinq modèles** de régression. Chacun repose sur des méthodes mathématiques et algorithmiques distinctes pour prédire des valeurs continues à partir des caractéristiques fournies. Ces modèles permettent de comprendre et de modéliser les relations entre les variables explicatives et la variable cible, offrant des outils adaptés à divers besoins de **prévision**, d'**estimation** ou d'**analyse quantitative**.

Nous présenterons par la suite une comparaison détaillée de plusieurs modèles de machine learning, notamment la régression linéaire (**LM**), la régression linéaire robuste (**RLM**), les machines à vecteurs de support (**SVM**), la régression Elastic Net (**glmnet**) et les Gradient Boosting Machines (**GBM**). Chaque modèle est décrit en termes de fonctionnement, d'avantages et d'inconvénients.

0.5.1.1 Régression Linéaire (LM)

La régression linéaire est un modèle statistique simple utilisé pour prédire une variable cible continue en fonction de variables explicatives. Elle repose sur la relation linéaire entre les variables et vise à minimiser la somme des erreurs quadratiques entre les valeurs observées et prédites.

Étapes du fonctionnement :

1. Déterminer la relation entre les variables **indépendantes** et la variable **cible**.
2. Construire un modèle linéaire basé sur l'équation :

$$\hat{y} = \beta_0 + \sum_{i=1}^p \beta_i x_i$$

où β_0 est l'interception et β_i sont les coefficients associés aux variables explicatives x_i .

3. Ajuster les coefficients en minimisant la somme des erreurs quadratiques (**méthode des moindres carrés**).
4. Évaluer la performance du modèle à l'aide de métriques comme le R^2 ou l'**erreur quadratique moyenne**.

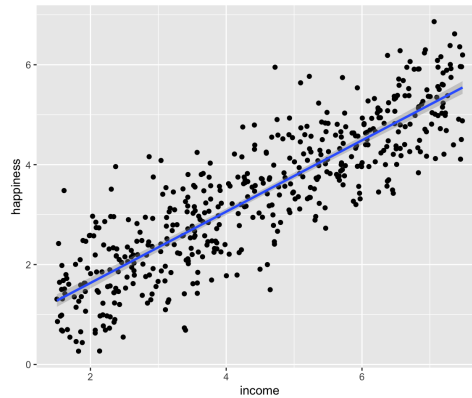


FIGURE 20 – Illustration de la régression linéaire.

Avantages :

- Simple à comprendre et rapide à mettre en œuvre.
- Fournit une interprétation claire des relations entre les variables.

Inconvénients :

- Sensible aux valeurs aberrantes et aux données non linéaires.
- Requiert des hypothèses comme la normalité des résidus et l'absence de multicollinéarité.

0.5.1.2 Régression Linéaire Robuste (RLM)

La régression linéaire robuste est une version améliorée de la régression linéaire conçue pour réduire l'influence des valeurs aberrantes. Elle utilise une fonction de perte robuste pour minimiser les résidus extrêmes.

Étapes du fonctionnement :

1. Identifier les valeurs aberrantes dans les données à l'aide d'une mesure robuste comme la médiane.
2. Construire un modèle basé sur une fonction de perte robuste $\rho(u)$ qui réduit l'impact des résidus extrêmes.
3. Ajuster les coefficients β en utilisant une méthode d'estimation itérative (comme M-estimation).
4. Évaluer la stabilité et la performance du modèle sur les données.

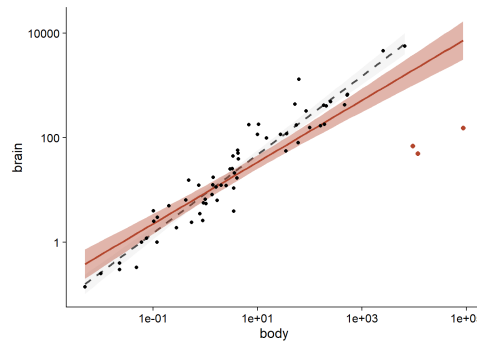


FIGURE 21 – Illustration de la régression linéaire robuste (RLM).

Avantages :

- Plus résiliente aux valeurs aberrantes que la régression linéaire ordinaire.
- Utile pour les données bruitées ou avec des erreurs systématiques.

Inconvénients :

- Peut être plus complexe et nécessiter un réglage des paramètres pour la fonction de perte.
- Moins performant si les données ne contiennent pas d'aberrations significatives.

0.5.1.3 Machines à Vecteurs de Support (SVM)

Le SVM est un algorithme d'apprentissage supervisé utilisé pour les tâches de classification et de régression. Il vise à maximiser la marge entre les données de différentes classes en utilisant des vecteurs supports.

Étapes du fonctionnement :

1. Identifier l'hyperplan optimal qui sépare les données en classes distinctes.
2. Maximiser la marge entre l'hyperplan et les points les plus proches (vecteurs supports).
3. Utiliser une fonction noyau (linéaire, polynomial, RBF) pour traiter les données non linéaires.
4. Prédire les étiquettes des nouvelles données en fonction de leur position par rapport à l'hyperplan.

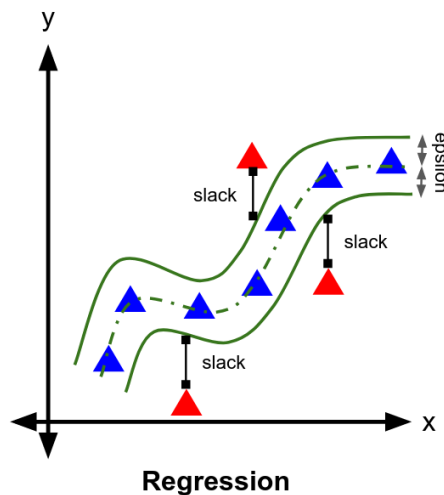


FIGURE 22 – Illustration des Machines à Vecteurs de Support (SVM).

Avantages :

- Performant pour des données de petite dimension et des marges bien définies.
- Flexible grâce aux différents noyaux pour traiter les non-linéarités.

Inconvénients :

- Moins efficace pour les grands ensembles de données.
- Nécessite un réglage précis des paramètres, comme C et le type de noyau.

0.5.1.4 Régression Elastic Net (`glmnet`)

`glmnet` est une méthode de régression pénalisée qui combine les régularisations L1 (lasso) et L2 (ridge) pour gérer les problèmes de colinéarité et de sélection de variables.

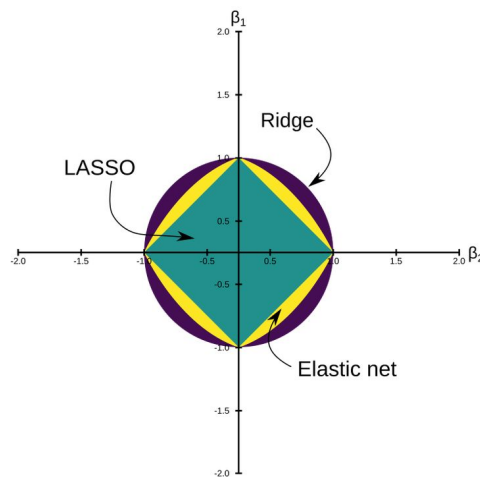
Étapes du fonctionnement :

1. Formuler une fonction de coût pénalisée :

$$\text{minimize} \quad \frac{1}{2N} \|y - X\beta\|_2^2 + \lambda (\alpha \|\beta\|_1 + (1 - \alpha) \|\beta\|_2^2)$$

où α contrôle le ratio entre L1 et L2.

2. Ajuster les coefficients β pour minimiser cette fonction de coût.
3. Sélectionner les variables pertinentes grâce à la pénalisation L1.
4. Valider les performances à l'aide de la validation croisée pour choisir λ .

FIGURE 23 – Illustration de la régression Elastic Net (`glmnet`)**Avantages :**

- Efficace pour les problèmes de haute dimension.
- Combine sélection de variables et régularisation.

Inconvénients :

- Nécessite une validation pour choisir les hyperparamètres.
- Peut réduire l'interprétabilité des coefficients.

0.5.1.5 Gradient Boosting Machines (GBM)

Le GBM est un modèle d'ensemble basé sur des arbres de décision qui optimise les prédictions en construisant des arbres séquentiels pour corriger les erreurs des prédictions précédentes.

Étapes du fonctionnement :

1. Initialiser un modèle faible (un arbre de décision).
2. Calculer les résidus entre les prédictions et les valeurs réelles.
3. Ajuster un nouvel arbre pour minimiser ces résidus.
4. Répéter le processus pour un nombre défini d'arbres en pondérant les prédictions.
5. Agréger les prédictions des arbres pour obtenir le résultat final.

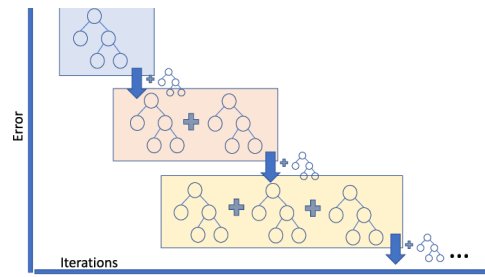


FIGURE 24 – Illustration des Gradient Boosting Machines (GBM).

Avantages :

- Performant pour les données complexes et non linéaires.
- Flexible et capable de gérer des données de différents types.

Inconvénients :

- Sensible au surajustement si mal paramétré.
- Plus lent à entraîner que les modèles simples.

0.5.2 Le jeu de données de prédiction des prix des maisons

Le dataset **House Pricing** est conçu pour prédire le **prix de vente des maisons** en fonction de leurs caractéristiques. Il contient des données sur **545** observations et **13** variables, dont une variable cible (**price**) et 12 variables explicatives offrant une combinaison d'attributs numériques et catégoriels. Ce dataset est disponible sur **Kaggle**.

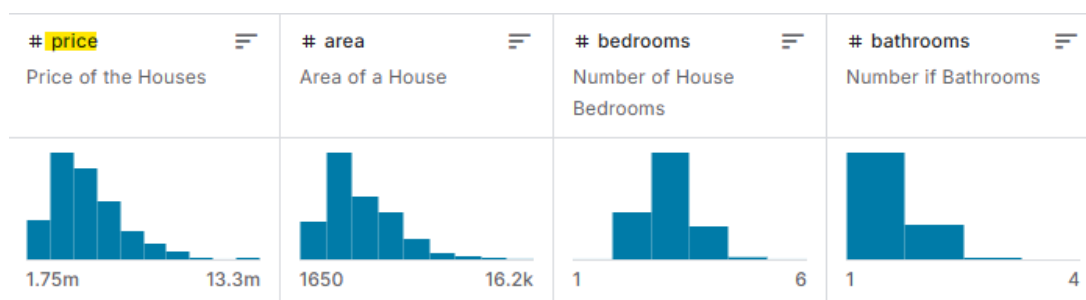


FIGURE 25 – Le jeu de données de la prédiction des prix des maisons

Le dataset contient les types de données suivants :

- **Variables numériques :**

- *price* : Prix de vente des maisons.
- *area* : Surface totale en pieds carrés.
- *bedrooms* : Nombre de chambres.

- **Variables catégorielles :**

- *bathrooms* : Nombre de salles de bain.
- *stories* : Nombre d'étages.
- *mainroad* : Proximité d'une route principale (*oui/non*).
- *guestroom* : Présence d'une guestroom (*oui/non*).
- *basement* : Présence d'un sous-sol (*oui/non*).
- *hotwaterheating* : Présence de chauffage à eau chaude (*oui/non*).
- *airconditioning* : Présence de climatisation (*oui/non*).
- *parking* : Nombre de places de parking.
- *prefarea* : Localisation dans une zone préférée (*oui/non*).
- *furnishingstatus* : État d'ameublement (*furnished/semi-furnished/unfurnished*).

0.5.2.1 Observations importantes

- **Données manquantes** : Aucune donnée manquante n'est observée dans ce dataset.

```
column: price      Missing values: 0
column: area       Missing values: 0
column: bedrooms   Missing values: 0
column: bathrooms  Missing values: 0
column: stories    Missing values: 0
column: mainroad   Missing values: 0
column: guestroom  Missing values: 0
column: basement   Missing values: 0
column: hotwaterheating Missing values: 0
column: airconditioning Missing values: 0
column: parking    Missing values: 0
column: prefarea   Missing values: 0
column: furnishingstatus Missing values: 0
```

FIGURE 26 – Absence des données manquantes sur le jeu de données

- **Colonnes catégorielles** : Certaines variables catégorielles doivent être transformées en facteurs pour faciliter leur analyse.

0.5.3 Pré-traitement du jeu de données

0.5.3.1 Élimination des Variables Non Pertinentes

Certaines variables peuvent ne pas être pertinentes pour le modèle et doivent être éliminées pour éviter l'introduction de bruit :

- **Affichage de la matrice de corrélation** L'affichage de la matrice de corrélation permet d'identifier les redondances et d'examiner les interactions entre les caractéristiques des maisons.

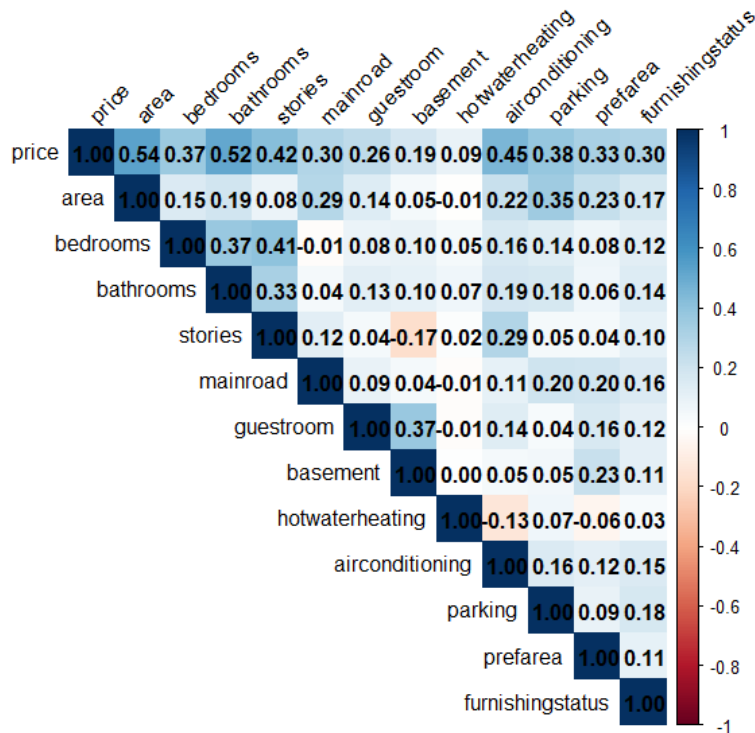


FIGURE 27 – Matrice de corrélation.

On observe que **area** affiche une forte corrélation positive avec **price** (**0.54**), ce qui signifie que la surface est un facteur clé dans la détermination du prix. D'autres variables comme **bedrooms** (**0.35**) et **bathrooms** (**0.33**) montrent également une corrélation positive notable, indiquant leur impact direct mais modéré sur le prix. En revanche, des variables comme **guestroom**, **basement**, et **airconditioning** ont des corrélations plus faibles (entre **0.10** et **0.19**), suggérant qu'elles jouent un rôle moins significatif. Certaines variables comme **hotwaterheating** et **furnishingstatus** montrent des corrélations très faibles, proches de zéro, indiquant qu'elles ont peu d'impact linéaire sur le prix.

- **Traitement des variables catégorielles :** Convertir les variables catégorielles en variables numériques à l'aide de la commande **factor**. Après la conversion, voici le résultat :

```

'data.frame':  545 obs. of  13 variables:
 $ price      : int  13300000 12250000 12250000 12215000 11410000 10850000 10150000
10150000 9870000 9800000 ...
 $ area       : int   7420 8960 9960 7500 7420 7500 8580 16200 8100 5750 ...
 $ bedrooms   : int    4 4 3 4 4 3 4 5 4 3 ...
 $ bathrooms  : int    2 4 2 2 1 3 3 3 1 2 ...
 $ stories    : int    3 4 2 2 2 1 4 2 2 4 ...
 $ mainroad   : Factor w/ 2 levels "no","yes": 2 2 2 2 2 2 2 2 2 2 ...
 $ guestroom  : Factor w/ 2 levels "no","yes": 1 1 1 1 2 1 1 1 2 2 ...
 $ basement   : Factor w/ 2 levels "no","yes": 1 1 2 2 2 2 1 1 2 1 ...
 $ hotwaterheating : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ airconditioning : Factor w/ 2 levels "no","yes": 2 2 1 2 2 2 2 2 1 2 2 ...
 $ parking    : int    2 3 2 3 2 2 2 0 2 1 ...
 $ prefarea   : Factor w/ 2 levels "no","yes": 2 1 2 2 1 2 2 1 2 2 ...
 $ furnishingstatus: Factor w/ 3 levels "unfurnished",...: 3 3 2 3 3 2 2 1 3 1 ...

```

FIGURE 28 – Type des variables après conversion.

0.5.3.2 Normalisation ou Standardisation des Variables

Les modèles d'apprentissage automatique sont souvent sensibles à l'échelle des variables. Il est donc recommandé de normaliser ou de standardiser les données :

- **Normalisation** : Centrer les données (soustraction de la moyenne) et réduire (division par l'écart-type).

```

> preProc <- preProcess(data[, num_vars],
  method = c("center", "scale"))

```

- **Standardisation** : Mettre à l'échelle les données entre 0 et 1 si nécessaire.

0.5.4 Comparaison entre les modèles

Le tableau ci-dessous présente la comparaison des cinq modèles de machine learning en termes de deux métriques de performance : **RMSE** (Root Mean Square Error) et R^2 (coefficient de détermination).

Modèle	RMSE	R^2
Régression Linéaire Robuste (RLM)	5.25×10^6	0.687
Régression Linéaire (LM)	5.25×10^6	0.683
Machines à Vecteurs de Support (SVM)	5.25×10^6	0.638
Régression Elastic Net (glmnet)	1.15×10^6	0.637
Gradient Boosting Machines (GBM)	1.17×10^6	0.667

TABLE 2 – Comparaison des modèles de machine learning en termes de RMSE et R^2

0.5.5 Modèle Choisi : Régression Linéaire Robuste (RLM)

Après analyse du tableau 2, le meilleur modèle identifié est la **Régression Linéaire Robuste (RLM)**. Ce choix repose sur ses performances équi-

brées en termes de **RMSE** et de R^2 . Avec un R^2 de **0.687**, le modèle explique une proportion importante de la variance de la variable cible (*price*), et son RMSE de 5.25×10^6 reflète une précision acceptable dans les prédictions.

La valeur de **RMSE** (*Root Mean Square Error*) représente l'écart moyen entre les valeurs réelles des prix des maisons et les valeurs prédites, exprimé dans les mêmes unités que la variable cible (*price*, en devises monétaires). Dans ce cas, un **RMSE** de 5.25×10^6 signifie que les prédictions du modèle s'écartent en moyenne de 5.25 millions d'unités monétaires du prix réel. Bien que cette valeur puisse sembler élevée, elle est cohérente avec l'échelle des prix dans le dataset, où les valeurs s'étendent de 1.75×10^6 à 13.3×10^6 . Voici les étapes suivies pour l'entraînement de ce modèle :

0.5.5.1 Chargement des Bibliothèques et des Données

Dans cette étape, les bibliothèques nécessaires sont chargées pour le prétraitement et la modélisation des données. Les données sont ensuite chargées à partir d'un fichier CSV.

```
> data <- read.csv("Housing.csv")
> str(data)
> colSums(is.na(data))
```

0.5.5.2 Division des Données en Ensembles d'Entraînement et de Test

Les données sont divisées en un ensemble d'entraînement (**80 %**) et un ensemble de test (**20 %**) afin de tester le modèle sur des données qu'il n'a pas vues pendant l'entraînement.

```
> set.seed(123)
> trainIndex <- createDataPartition(data$price,
p = 0.8, list = FALSE)
> train_data <- data[trainIndex, ]
> test_data <- data[-trainIndex, ]
```

0.5.5.3 Entraînement du Modèle de Régression Linéaire

Le modèle de régression linéaire est entraîné sur l'ensemble d'entraînement pour prédire la variable cible, ici le prix des maisons.

```
> model_rlm <- rlm(price ~ ., data = train_data)
```

0.5.5.4 Description du Modèle de Régression Linéaire "RLM"

Le modèle de régression linéaire "RLM" est une méthode statistique utilisée pour prédire une variable continue en fonction de variables explicatives (prédicteurs). Le modèle repose sur l'équation suivante :

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

où : - y est la variable dépendante (ici, le prix des maisons). - β_0 est l'ordonnée à l'origine (intercept). - $\beta_1, \beta_2, \dots, \beta_n$ sont les coefficients de régression des variables indépendantes $x_1, x_2, \dots, x_n$. - ϵ est l'erreur aléatoire (résidu), qui capture les influences non modélisées par les variables explicatives.

Voici un summary du modèle **RLM** :

```
call: rlm(formula = price ~ ., data = train_data)
Residuals:
    Min       1Q   Median       3Q      Max
-1.337380 -0.280353  0.005573  0.264751  2.683683

Coefficients:
              value      Std. Error t value
(Intercept)   -0.6946      0.0689  -10.0884
area           0.2736      0.0256   10.6861
bedrooms       0.0253      0.0260    0.9746
bathrooms      0.2507      0.0255    9.8315
stories        0.2240      0.0269    8.3360
mainroadyes    0.2241      0.0680    3.2974
guestroomyes   0.1297      0.0649    1.9997
basementyes    0.1943      0.0549    3.5427
hotwaterheatingyes 0.3910      0.1029    3.7991
airconditioningyes 0.4049      0.0532    7.6167
parking        0.0935      0.0249    3.7551
prefareayes    0.3088      0.0566    5.4591
furnishingstatussemi-furnished 0.1909      0.0540    3.5379
furnishingstatusfurnished 0.2388      0.0624    3.8278

Residual standard error: 0.4114 on 424 degrees of freedom
```

FIGURE 29 – Type des variables après conversion.

Ce tableau présente les résultats d'un modèle de régression robuste (RLM) visant à prédire la variable cible **price** en fonction de plusieurs variables explicatives. Les résidus montrent une distribution symétrique autour de la médiane proche de zéro (**0.005573**), ce qui indique que le modèle est bien ajusté, malgré quelques écarts observés (résidus allant de -1.337380 à 2.683683). Les coefficients des variables montrent leur contribution relative à la prédiction de **price**, avec leurs valeurs, erreurs standards, et les statistiques t. Par exemple, les variables comme **area** (**coefficient = 0.2736**, **t = 10.6861**) et **bathrooms** (**coefficient = 0.2507**, **t = 9.8315**) ont une forte influence positive sur **price**, tandis que des variables catégoriques comme

`furnishingstatussemi-furnished` et `furnishingstatusfurnished` sont également significatives. La faible erreur résiduelle standard (**0.4114**) indique une bonne capacité prédictive du modèle.

0.5.5.5 Évaluation du Modèle

Les performances du modèle sont évaluées en utilisant des métriques telles que l'erreur quadratique moyenne (**RMSE**), et le coefficient de détermination R^2 .

Pour une meilleure visualisation des prédictions, un graphique est ainsi tracé pour comparer les valeurs réelles et les prédictions, avec une ligne de référence $y = x$ pour visualiser la performance du modèle. Pour ce faire, nous procédons comme suit :

```
> predictions <- predict(model_rlm, newdata = test_data)
> performance <- postResample(pred = predictions,
  obs = test_data$price)
> plot(test_data$price, predictions,
  main = "Pr dictions vs R els",
  xlab = "Prix R els",
  ylab = "Pr dictions",
  col = "blue",
  pch = 16)
  abline(0, 1, col = "red")
```

Voici le graphe qui montre une comparaison entre les valeurs prédites et les valeurs réelles :

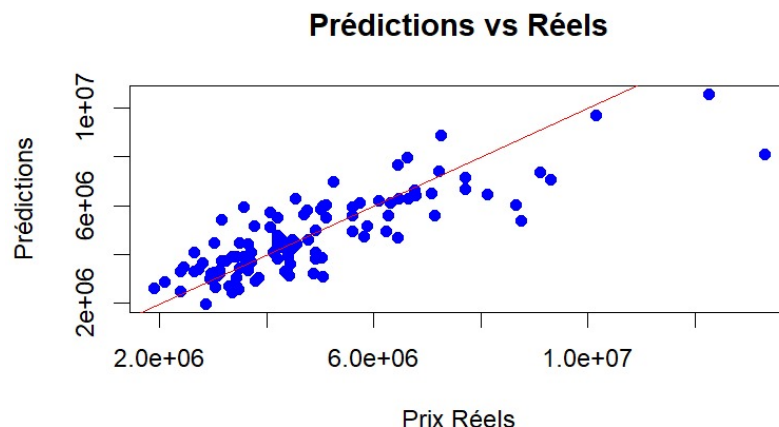


FIGURE 30 – Evaluation du modèle

Ce graphique montre la comparaison entre les **valeurs réelles** (en abscisse) et les **valeurs prédites** (en ordonnée) par le modèle de régression. La ligne rouge correspond à la ligne idéale $y = x$, où les prédictions seraient parfaitement égales aux valeurs réelles.

- **Alignement autour de la ligne rouge** : La plupart des points bleus sont proches de la ligne rouge, ce qui indique que le modèle réalise des prédictions assez précises.
- **Dispersion des points** : On observe une certaine dispersion des points, particulièrement pour des valeurs réelles élevées, ce qui suggère que le modèle est moins précis pour prédire des prix très élevés.
- **Biais éventuel** : La distribution des points semble légèrement asymétrique pour les valeurs réelles moyennes à élevées, ce qui pourrait signifier un biais dans le modèle (par exemple, une tendance à sous-estimer ou à surestimer les prix dans certaines plages).

0.5.5.6 Interprétation des résultats de RLM

Le modèle de **Régression Linéaire Robuste (RLM)** a montré une performance raisonnable dans la prédiction de la variable cible. Avec un **erreur quadratique moyenne (RMSE)** de 5.25×10^6 , le modèle présente une précision correcte dans les prédictions globales, bien qu'il puisse être encore optimisé pour des valeurs extrêmes. Le coefficient de détermination (R^2) de **0.687** indique que le modèle explique environ **68.7% de la variance** des données, ce qui est satisfaisant pour un modèle robuste. Cependant, des améliorations, comme l'ajout de variables explicatives pertinentes ou l'exploration de modèles non linéaires, pourraient être envisagées pour augmenter cette précision et réduire les erreurs dans certaines plages de valeurs.

0.5.6 Conclusion

En conclusion, la partie régression a permis d'explorer et d'évaluer plusieurs modèles, dont la **Régression Linéaire (LM)**, la **Régression Linéaire Robuste (RLM)**, les **Machines à Vecteurs de Support (SVM)**, la **Régression Elastic Net (glmnet)** et les **Gradient Boosting Machines (GBM)**. Parmi ces modèles, la **Régression Linéaire Robuste (RLM)** s'est distinguée grâce à son équilibre entre robustesse et performance, en particulier dans la gestion des valeurs aberrantes, avec un coefficient de détermination (R^2) de **0.687** et un RMSE de 5.25×10^6 .

Cela s'explique par le fait que le **RLM** est conçu pour être **robuste face aux valeurs aberrantes** et aux **distributions non idéales**, contrairement aux autres modèles plus sensibles à ces limitations. Cette analyse a également mis en lumière l'importance du **prétraitement des données**, notamment la **normalisation** et l'**élimination des variables moins pertinentes**, pour garantir des **performances optimales**. Ces résultats soulignent que le **choix du modèle** dépend des **spécificités des données** et des **objectifs analytiques**, tout en mettant en évidence la nécessité d'une **évaluation approfondie** pour guider ce choix.

Conclusion Générale

En conclusion, ce rapport a proposé une étude de cas détaillée sur l'utilisation du package **caret**. L'accent a été mis sur l'application pratique de différents modèles de machine learning sur des jeux de données appropriés, en tenant compte des méthodes de classification et de régression. À travers cette étude, nous avons exploré les principales fonctionnalités du package, y compris l'entraînement de modèles avec validation croisée et la recherche d'hyperparamètres optimaux.

Une comparaison des performances des modèles a été effectuée en utilisant plusieurs métriques adaptées, comme la ROC, DET, le RMSE et le R², permettant ainsi d'évaluer l'efficacité des modèles dans des tâches concrètes. Les résultats obtenus ont montré que le package **caret** est un outil puissant pour optimiser et évaluer les modèles de machine learning, tout en facilitant leur utilisation, même pour des utilisateurs débutants ou intermédiaires.

En conclusion, l'étude des algorithmes de classification et de régression construits autour du package **caret** a permis de mettre en évidence leur efficacité et leur capacité à résoudre une large gamme de problèmes dans le domaine du machine learning. Cette étude illustre non seulement la flexibilité du package, mais aussi son rôle essentiel dans la création de modèles prédictifs robustes et performants.

Bibliographie

- [1] Max Kuhn. The caret Package. Available at : <https://topepo.github.io/caret/>
- [2] Yasser H. Housing Prices Dataset. Available at : <https://www.kaggle.com/datasets/yasserh/housing-prices-dataset>
- [3] Rabie El Kharoua. Alzheimer's Disease Dataset. Available at : <https://www.kaggle.com/datasets/rabieelkharoua/alzheimers-disease-dataset>
- [4] IBM. Decision Trees. Available at : <https://www.ibm.com/fr-fr/topics/decision-trees>
- [5] IBM. XGBoost : Extreme Gradient Boosting. Available at : <https://www.ibm.com/fr-fr/topics/xgboost>
- [6] Data Analytics Post. Lexique : Support Vector Machine (SVM). Available at : <https://dataanalyticspost.com/Lexique/svm/>
- [7] Novus Afk. Linear Regression : An Overview. Available at : https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd
hyperref
- [8] Ujang Riswanto. How Robust Regression Solves Real-World Data Challenges. Available at : <https://ujangriswanto08.medium.com/how-robust-regression-solves-real-world-data-challenges-df96077478dd>